

Linux Privilege Escalation
CVE-2016-5195 ("Dirty COW")
A critical memory management vulnerability

Theodor Negrescu
Master's Student, University of Bucharest, Romania

2026-02-06

Introduction

- ▶ **CVE-2016-5195** ("Dirty COW") was a severe privilege escalation vulnerability in the Linux kernel, allowing attackers to write to sensitive system files.
- ▶ **Read access to any file → Writable**
- ▶ This made it trivial to escalate privileges by modifying files such as `setuid` binaries, `/etc/passwd`, `/etc/sudoers`, cron jobs, and more.
- ▶ Due to a race condition caused by writes through `/proc/self/mem` to read-only memory mappings while another thread was calling `madvise(MADV_DONTNEED)`, the COW mechanism could be bypassed to write to the backing page cache page.

Intended COW Behavior

- ▶ An important thing to note is that writing to mappings of read-only files is not forbidden in Linux. However, such writes are not supposed to be written back to the underlying file.
- ▶ The COW mechanism should create a private copy of any page which is written to. This should ensure modifications are isolated. Modified pages are called "dirty".
- ▶ COW is only enabled when using the `MAP_PRIVATE` flag to `mmap`. `MAP_SHARED` indicates writes should be to the actual underlying file, but you can only use `MAP_SHARED` on file descriptors opened read-write.
- ▶ `MAP_PRIVATE` is not supposed to allow modifying files. Dirty pages are discarded without write-back when the process exits.
- ▶ But the Dirty COW exploit could make writes to `MAP_PRIVATE` mappings end up in the actual page cache.

The importance of `/proc/self/mem` and `ptrace`

- ▶ The vulnerability fundamentally relies on the way the debugging interfaces `/proc/self/mem` and `ptrace` handle writes.
- ▶ Attackers never directly write to the mapped area. The COW mechanism would properly create a dirty page in that case.
- ▶ Linux provides debugging interfaces which allows reading and writing memory indirectly, with applications to debugging and crash reporting. They can also be used for accessing other processes, which is not necessary for this exploit.
- ▶ There is a small deviation from normal write behavior. For debugging convenience, you can write to read-only memory. For example, you may want to patch code at runtime.
- ▶ This is not intended to bypass security. Writes to COW pages should still trigger a dirty page to be created as usual. Memory isolation **should** still be maintained.

The `__get_user_pages` kernel function

- ▶ While handling indirect memory writes, the kernel calls the `__get_user_pages` to find the destination page and pin it into memory.
- ▶ To find the page, a retry loop is used internally, with flags containing the bits `FOLL_WRITE` and `FOLL_FORCE`, indicating intent to write, and to ignore read-only permissions.
 1. Call `follow_page_mask(..., flags, ...)` to walk the process page table to find the page.
 2. If `follow_page_mask` failed, call `faultin_page(..., &flags, ...)` to trigger page fault handling to bring the page into the process address space, and then retry.
 3. Return the page to the caller, which in this case will write to it.

The `faultin_page` flags "trick"

- ▶ After creating a dirty copy for a read-only COW page, `faultin_page` **clears the `FOLL_WRITE` in flags, which is passed by reference.**
- ▶ This is because calling `follow_page_mask` would fail to find the dirty page, since it is marked read-only, causing an infinite retry loop.
- ▶ Because the dirty copy is private to the process, it is safe for `__get_user_pages` to return it, even though it is read-only, since the write will be isolated.
- ▶ But implementing this by clearing the `FOLL_WRITE` bit opens up a critical race condition.
- ▶ The process page table can be modified in between calling `faultin_page` and the next call to `follow_page_mask`.

The unintended code flow

- ▶ `__get_user_pages` calls `faultin_page`, creating a dirty page and clearing the `FOLL_WRITE` bit.
- ▶ Another user thread calls `madvise(MADV_DONTNEED)`, which is intended to inform the kernel that a page will not be needed soon and can be discarded. **This is the race condition.**
- ▶ The next retry loop iteration calls `follow_page_mask` again. This is supposed to find the dirty page, but it is gone.
- ▶ The modified flags are used for a second call to `faultin_page`.
- ▶ `faultin_page` now thinks it is safe to use the backing page cache page, since there doesn't seem to be a need for isolation, with `FOLL_WRITE` cleared.
- ▶ The final call to `follow_page_mask` finds the shared page which was mapped into the process address space. It is returned, and written to, modifying the underlying file.

Impact and Exploitation

- ▶ Running the exploit in a loop succeeds in seconds.
- ▶ Memory isolation is bypassed. Any file the attacker has read access to can be modified.
- ▶ Besides trivial exploits such as modifying setuid binaries or `/etc/passwd`, many complex exploits are possible. System binaries could be backdoored, configuration files could be modified to disable security, cron jobs could be modified, and more.
- ▶ The exploit was discovered by security researcher Phil Oester in October 2016, after he extracted the exploit from a packet capture of a compromised web server.
- ▶ Linus Torvalds released a patch on October 20 2016. Many vendors provided live patching utilities to apply the patch without rebooting. It is still considered one of the most severe privilege escalation vulnerabilities in Linux ever.

The Fix

- ▶ Instead of clearing `FOLL_WRITE`, a new bit was introduced, `FOLL_COW`, to indicate that a dirty page has been created, and read-only permissions can safely be ignored.
- ▶ When `FOLL_COW` is set, `follow_page_mask` will return the dirty page, but only after checking that it is indeed a dirty page, not a shared page from the page cache.
- ▶ If the dirty page is discarded by `madvise(MADV_DONTNEED)`, this only causes another retry loop iteration. No code flow exists which leads to returning a shared page for writing.
- ▶ Instead of going to the page cache, writes are correctly isolated to a dirty page. When the process exits, private dirty pages are simply discarded without being written back to disk.
- ▶ Intended behavior is restored.

Except...

- ▶ A second exploit, CVE-2017-1000405 ("Huge Dirty COW") was discovered in 2017 by researchers from Bindency.
- ▶ A slightly different bug in handling of THP (Transparent Huge Pages) can cause a page to be incorrectly marked as dirty, defeating the security check performed when FOLL_COW is set.
- ▶ Thankfully, THP is not used when mapping ordinary files.
- ▶ Huge Dirty COW can affect the huge zero page, which affects processes which read from newly allocated memory before writing to it, and it can be used to defeat sealing on shmem shared memory, which could be part of an exploit chain (such as against browsers with tab isolation), but it is not nearly as severe as the original Dirty COW.
- ▶ This second exploit was discovered by analysis, and there is no indication it has ever been exploited in the wild.

Lessons

- ▶ **Keep logic explicit.** Clearing `FOLL_WRITE` instead of having a dedicated `FOLL_COW` bit introduced the vulnerability.
- ▶ **Secure concurrent code is hard.** The original code assumed the dirty page would still be there on the next iteration. But the page could be discarded by another thread.
- ▶ **Debugging features are an attack surface.** The exploit relies on writing to memory through debugging interfaces, and on the handling of read-only pages on top of that.
- ▶ **Security is never perfect.** It is extremely difficult to write secure code. Even though provably secure kernels exist, Linux dominates in the server market due to performance and feature advantages. Exploits as severe as Dirty COW probably exist in current Linux kernels. Defense in depth is important to protect against zero-day exploits. Users should keep backups and use 2FA to mitigate the damage from compromised devices.

Conclusion

This has been a brief presentation on the Dirty COW exploit, CVE-2016-5195, for the Operating Systems: Design and Security course at FMI, University of Bucharest.

The full paper is available online at

https://github.com/theo543/OSDS_Paper_CVE-2016-5195.

Thank you for your time and attention!